

Dilemma van de dubbelagent

- **Tijdslimiet:** 12 minuten.
- **Opslag:** 5 GB
- **Omgeving:** één GPU (≈16 GB VRAM), geen internet
- **Grootte van de oplossing:** `solution.ipynb` ≤ 1 MB
- **Baselinescore:** 0

In het nationale AI-centrum in Astana bekijken ze de "breinen" van verschillende computermodellen — Model R (een ResNet-18) en Model V (een ViT-Tiny). Op dit moment leveren beide modellen perfect werk: ze behalen een nauwkeurigheid van 100% en zijn het over elke afzonderlijke afbeelding eens. In plaats van ze op een dwangbuis te gooien en hun brein open te snijden, proberen ze wat vreedzamer: de afbeeldingen een klein beetje veranderen om de twee modellen een net wat ander resultaat te geven.



1. Opdracht

Twee vooraf getrainde afbeeldingsclassificatoren bekijken dezelfde afbeelding. Op de afbeeldingen die voor deze opdracht worden verstrekt, behalen beide classificatoren een nauwkeurigheid van 100%.

- **Model R:** `torchvision.models.resnet18` (een CNN, ResNet18).
- **Model V:** `timm's vit_tiny_patch16_224` (een Transformer, ViT-Tiny).

Je opdracht is om voor elke afbeelding een kleine wijziging („perturbatie”) te maken, zodat de twee modellen het oneens zijn. Voor elke afbeelding moet je **twee verschillende** perturbaties maken:

- **Type A:** nadat deze is toegevoegd, classificeert Model R de afbeelding nog steeds correct, maar classificeert Model V deze onjuist.
- **Type B:** nadat deze is toegevoegd, classificeert Model V de afbeelding nog steeds correct, maar classificeert Model R deze onjuist.

Elke perturbatie moet *klein* genoeg zijn om moeilijk waarneembaar te zijn. Kleinere perturbaties leveren een hogere score op (zie Sectie 5). De perturbatie wordt rechtstreeks op pixelniveau op de oorspronkelijke afbeelding toegepast.

2. Openbare data

Bij de opdracht wordt een verzameling afbeeldingen verstrekt, ingedeeld in twee splits — `train` (100 afbeeldingen) en `test_public` (100 afbeeldingen) — elk met afbeeldingen van uiteenlopende resoluties. Alle afbeeldingen zijn afkomstig uit de 1000 klassen van ImageNet-1K en zowel Model R als Model V behaalt een nauwkeurigheid van 100% op beide splits.

De volgende bestanden worden verstrekt:

```
train/images/*.png      # 100 images in PNG format
train/labels.json       # maps each image index to its correct class
test_public/images/*.png # 100 images in PNG format
test_public/labels.json  # maps each image index to its correct class
```

Tijdens de beoordeling wordt je map `dataset/test_public/` transparant vervangen door twee verborgen verzamelingen afbeeldingen (`test_leaderboard_a` en `test_leaderboard_b`) voor de officiële scoreberekening. Elk daarvan bevat **100 afbeeldingen** in PNG-formaat en een labelbestand.

Opmerking: Voor deze opdracht zijn de labels in de testdatasets toegankelijk.

3. Uitvoerformaat

Voor elke afbeelding moet je twee bestanden produceren:

```
{index}_a.pt  # Type A perturbation
{index}_b.pt  # Type B perturbation
```

- $\{\text{index}\}$ (0, 1, 2, ...), komt overeen met de naam van de afbeelding in de datasets.
- Elk bestand is één tensor die is opgeslagen met `torch.save`. De vorm ervan moet $3 \times H \times W$ zijn, waarbij H en W overeenkomen met de **oorspronkelijke** resolutie van die afbeelding (niet 224×224).
- De code mag slechts één ZIP-bestand produceren, `submission.zip`. Plaats alle `.pt`-bestanden op het hoogste niveau van het ZIP-archief, zonder omsluitende map of submappen.

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

De notebook waarschuwt je als er problemen met het uitvoerformaat zijn.

4. Beperkingen

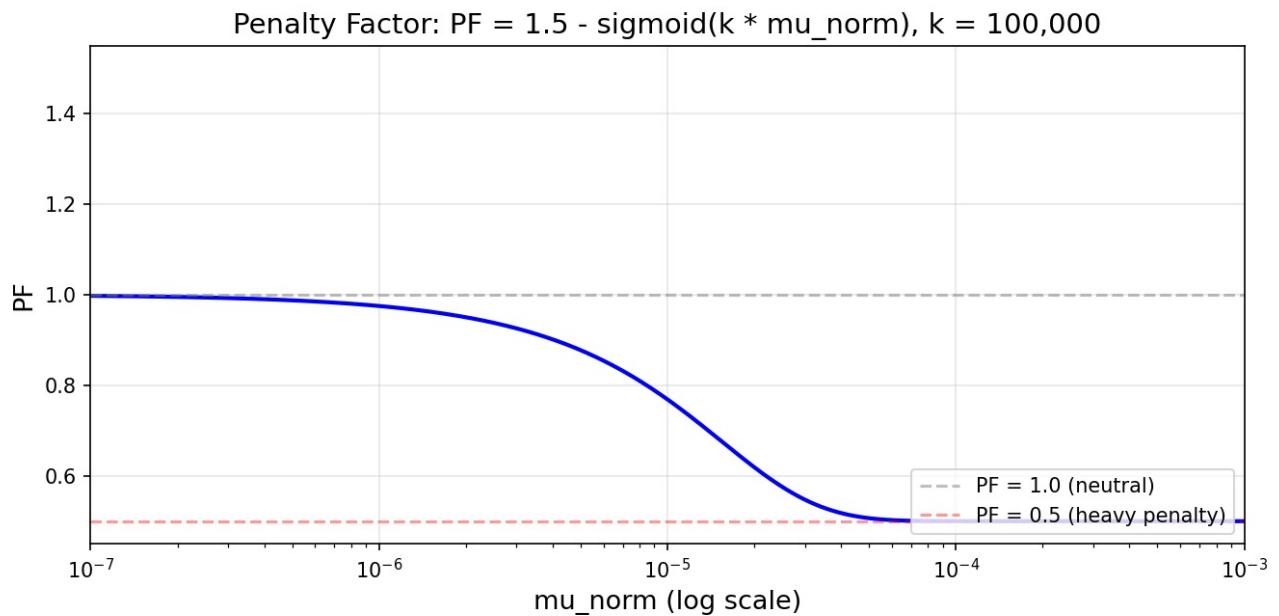
- **Modellen:** Je moet `torchvision.models.resnet18(pretrained=True)` en `timm.create_model('vit_tiny_patch16_224', pretrained=True)` gebruiken. Andere vooraf getrainde modellen zijn niet toegestaan.
- **Transformatiepipeline (afgedwongen tijdens de evaluatie):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. See Section 3 of `baseline.ipynb` voor details.
- **Resolutie van de perturbatie:** Moet overeenkomen met de **oorspronkelijke** resolutie van de ruwe afbeelding (niet 224×224). De tensor wordt aan de ruwe afbeelding toegevoegd vóór de transformatiepipeline.
- **Uitvoerformaat:** Alleen `.pt`-bestanden — geen PNG/JPG. Tensors worden aan de ruwe afbeelding toegevoegd en pixelwaarden worden vóór de preprocessing begrensd tot $[0, 1]$.
- **Bestandsnamen:** Platte lijst, strikt formaat `{index}_a.pt` / `{index}_b.pt`. Geen submappen in het zipbestand.
- **Bibliotheken:** `torch`, `torchvision`, `timm`.

5. Scoreberekening

De eindscore wordt als volgt berekend. Laat M het aantal afbeeldingen in de split zijn, $Score_A$ het aantal geslaagde perturbaties van Type A en $Score_B$ het aantal geslaagde perturbaties van Type B:

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF is een functie die is ontworpen om perturbaties met een hoge norm te bestraffen en zeer gevoelig te zijn nabij het prestatieplafond. De de functie is begrensd tot het bereik 0.5 tot 1. De volledige implementatie is te vinden in Sectie 8 van `solution.ipynb`.



Figuur: De curve van de straffunctie.

6. De inzending controleren

De notebook bevat controles die je waarschuwen als er opmaakproblemen zijn, in Sectie 7 van de notebook `solution.ipynb`.

7. Lokaal testen

`solution.ipynb` bevat een compleet, werkend voorbeeld. Het laadt de openbare data, beide modellen en de officiële scorer, en schrijft een ZIP-bestand voor de inzending. Lees het voordat je begint.

8. Indienen

- Sla je wijzigingen op in `solution.ipynb`.
- Open het tabblad Git in de linkerzijbalk van JupyterLab.
- **Stage** `solution.ipynb` (het pictogram + ernaast).
- Voer een commitbericht in en klik op **Commit**.
- Klik op het wolkpictogram met de omhoogwijzende pijl om te pushen.
- Ga terug naar deze wedstrijdpagina en klik op **Submit**.

Dien precies één bestand in, met de naam `solution.ipynb`.